

Sécurisation de l'infrastructure

Garantir Disponibilité - Intégrité - Confidentialité

Abdoullah ICHOU

Formation – 2 jours

Fil rouge

Chaque TP attaque un angle de la triade **DIC (CIA)** – de la machine hôte jusqu'à la supply chain.

- **TP0** : Déploiement stack legacy
- **TP1** : Audit & durcissement hôte
- **TP2** : Runtime Docker
- **TP3 / MSPR** : Supply chain & rapport

Node Exporter (:9100) → Prometheus (:9090) → Grafana (:3000)

Risques identifiés

- **Ports exposés** : SSH, Grafana, Prometheus visibles sur le réseau.
- **Credentials** : admin/admin sur Grafana, aucune auth sur Prometheus.
- **Volumes** : /var/run/docker.sock monté (accès root daemon).

Objectifs :

- `docker compose up -d`
- Vérifier Prometheus (Targets)
- Analyser les failles

```
docker compose up -d  
# Grafana : :3000
```

Livable

Pourquoi cette stack est-elle une "passoire" ? 3 risques + remédiations.

Penser comme l'attaquant (nmap)

```
sudo nmap -sV -p 22,3000,9090,9100 localhost
```

- Quels services sont visibles ?
- Analyse : OpenSSH vulnérable, accès Grafana, scraping Prometheus.

- **Isolation** : Forcer `ListenAddress 127.0.0.1`.
- **Utilisateur** : Créer devops + groupe sudo.
- **Authentification** : Clés **Ed25519**.
- **Sécurisation** : Désactiver `PasswordAuthentication` et `PermitRootLogin`.

Question

Pourquoi Ed25519 est-il supérieur à RSA ?

Filtrage de l'hôte

```
sudo ufw default deny incoming
sudo ufw allow ssh
sudo ufw enable
```

Limite critique

Docker modifie iptables et **contourne UFW**. Les ports conteneurs restent accessibles.

Isoler les conteneurs

- Supprimer les ports : publics dans Docker Compose.
- Point d'entrée unique sur le port 80/443 via `proxy_pass`.
- `/grafana/` → `Grafana:3000`

Bénéfice

Les services ne sont plus exposés directement sur le réseau.

Rendu attendu (TP0 + TP1)

- Scan `nmap` + analyse.
- Réponses aux questions (Missions 1.1 à 1.4).
- Captures d'écran : Grafana via reverse proxy, accès direct refusé.
- **Format** : Markdown ou PDF.

- Partage du **Kernel** de l'hôte.
- Ressources CPU/RAM illimitées par défaut.

Vecteurs d'évasion

-privileged ou montage de `/var/run/docker.sock` = compromission totale possible.

Limiter la surface d'attaque

- `cap_drop: ALL` (Moindre privilège).
- `no-new-privileges: true`.
- Limitation ressources (CPU/RAM).

```
# Exemple de durcissement Docker Compose
services:
  grafana:
    security_opt:
      - no-new-privileges:true
    deploy:
      resources:
        limits:
          cpus: '0.50'
          memory: 512M
```

Audit avec Trivy

```
trivy image prom/prometheus:v2.40.0
```

- **Remédiation** : Pinner les versions (pas de latest).
- **Secrets** : Utiliser un `.env` et l'ajouter au `.gitignore`.

```
docker run --rm --net host --pid host \  
-v /var/run/docker.sock:/var/run/docker.sock \  
docker/docker-bench-security
```

Livrable Final

Rapport professionnel résumant l'audit et fichiers corrigés (remédiation).

Architecture sécurisée validée !

- TP0-TP1 : Hôte durci
- TP2 : Runtime Docker sécurisé
- TP3 : Supply Chain & Audit final

Merci pour votre attention !